

Programozás Alapjai

Dr. Gergely Tamás
Dr. Kicsi András

Szegedi Tudományegyetem
Informatikai Intézet
Szoftverfejlesztés Tanszék

2025

(v1207)



Tartalomjegyzék

- 
- 1 Bemutatók**
 - Kurzus információk
 - Infrastruktúra szabályzatok
 - Mentorálás
 - 2 Python alapok**
 - Bevezető
 - Pénzváltás (1. verzió)
 - Pénzváltás (2. verzió)
 - 3 Alapfogalmak**
 - Alapfogalmak
 - Alapvető programelemek fogalmak
 - 4 Python feltételes ciklusok**
 - Röppálya számítás
 - Röppálya szimuláció
 - 5 Szoftverfejlesztési életciklus modellek**
 - A programozás fázisai
 - 6 Python sztringek**
 - Python sztringek
 - Basename meghatározása
 - 7 Vezérlési szerkezetek**
 - Bevezetés
 - Szekvenciális vezérlés
 - Függvények
 - 8 Python listák**
 - Python list és tuple adattípusok
 - Python listaszerű adattípusok műveletei
 - Basename és eltelt idő
 - 9 Vezérlési szerkezetek**
 - Szelekciós vezérlések
 - 10 Python iterációk**
 - Csúszóátlag számítása
 - 11 Vezérlési szerkezetek**
 - Ismétléses vezérlések
 - 12 Python halmazok, comprehension**
 - Python set és frozenset adattípusok
 - Tér legtávolabbi pontjai
 - 13 Vezérlési szerkezetek**
 - Ismétléses vezérlések
 - 14 Python dictionary, függvény mint típus**
 - Python dict adattípus
 - Python comprehension
 - Python function adattípus
 - 15 Vezérlési szerkezetek**
 - Eljárásvezérlés
 - 16 Python fájlkezelés**
 - Fájlfeldolgozás
 - 17 Vezérlési szerkezetek**
 - Eljárásvezérlés
 - 18 Python formátált IO**
 - Python formátált IO
 - 19 Adatkezelés, adattárolás**
 - Az adatkezelés szintjei
 - Az adattárolás jellemzői
 - 20 Python típusok**
 - Python numerikus típusok
 - 21 Absztrakt adattípusok**
 - Egyszerű adattípusok
 - Összetett adattípusok
 - 22 Python típusok**
 - 23 Absztrakt adattípusok**
 - Összetett adattípusok
 - 24 Tesztelés**
 - Tesztelés
 - Tesztelés Python-ban
 - A pytest
 - 25 Vizualizáció és egyéb modulok**
 - Fájlkezelő modulok
 - Vizualizáció
 - 26 More Python**
 - A Python program végrehajtásának folyamata
 - Függvényparaméterek és argumentumok
 - Python unpacking
 - Kivételkezelés
 - További Python típusok**
 - Python function adattípus

Tartalomjegyzék

- 
- 1 Bemutatók**
 - Kurzus információk
 - Infrastruktúra szabályzatok
 - Mentorálás
 - 2 Python alapok**
 - Bevezető
 - Pénzváltás (1. verzió)
 - Pénzváltás (2. verzió)
 - 3 Alapfogalmak**
 - Alapfogalmak
 - Alapvető programelemek fogalmak
 - 4 Python feltételes ciklusok**
 - Röppálya számítás
 - Röppálya szimuláció
 - 5 Szoftverfejlesztési életciklus modellek**
 - A programozás fázisai
 - 6 Python sztringek**
 - Python sztringek
 - Basename meghatározása
 - 7 Vezérlési szerkezetek**
 - Bevezetés
 - Szekvenciális vezérlés
 - Függvények
 - 8 Python listák**
 - Python list és tuple adattípusok
 - Python listáshoz adattípusok műveletei
 - Basename és eltelt idő
 - 9 Vezérlési szerkezetek**
 - Szelekciós vezérlések
 - 10 Python iterációk**
 - Csúszóátlag számítása
 - 11 Vezérlési szerkezetek**
 - Ismétléses vezérlések
 - 12 Python halmazok, comprehension**
 - Python set és frozenset adattípusok
 - Tér legtovábbi pontjai
 - 13 Vezérlési szerkezetek**
 - Ismétléses vezérlések
 - 14 Python dictionary, függvény mint típus**
 - Python dict adattípus
 - Python comprehension
 - Python function adattípus
 - 15 Vezérlési szerkezetek**
 - Eljárásvezérlés
 - 16 Python fájlkezelés**
 - Fájlkezelés
 - 17 Vezérlési szerkezetek**
 - Eljárásvezérlés
 - 18 Python formátált IO**
 - Python formátált IO
 - 19 Adatkezelés, adattárolás**
 - Az adatkezelés szintjei
 - Az adattárolás jellemzői
 - 20 Python típusok**
 - Python numerikus típusok
 - 21 Absztrakt adattípusok**
 - Egyszerű adattípusok
 - Összetett adattípusok
 - 22 Python típusok**
 - 23 Absztrakt adattípusok**
 - További Python típusok
 - Python function adattípus
 - Összetett adattípusok
 - 24 Tesztelés**
 - Tesztelés
 - Tesztelés Python-ban
 - A pytest
 - 25 Vizualizáció és egyéb modulok**
 - Fájlkezelő modulok
 - Vizualizáció
 - 26 More Python**
 - A Python program végrehajtásának folyamata
 - Függvényparaméterek és argumentumok
 - Python unpacking
 - Kivételkezelés

- A Neumann-architektúra szerint minden adatmozgás, ami nem a memória és a processzor között történik, I/O műveletnek számít.
- Mi egy kicsit máshonnan nézzük ezt: minden adatmozgás, aminek a forrása vagy célja a „programon kívül” esik, I/O műveletet igényel.
- Az I/O műveleteket a legtöbb programozási nyelv fájlkezeléssel oldja meg, ez alól a Python sem kivétel.
- Python-ban megkülönböztetünk
 - szöveges, karakter alapú fájlkezelést olyan esetekhez, amikor az adat az ember számára olvasható, értelmezhető kell legyen (pl. szövegfájlok, programozási nyelvek forrásfájljai, .csv-k, ...), és
 - bináris, bájt alapú fájlkezelést, amikor az adatot kifejezetten gépi feldolgozásra szánjuk (pl. kép-, hang- és videofájlok, lefordított programok, ...).
- Bináris fájlokkal most nem fogunk foglalkozni.

- A jelenlegi szoftverarchitektúrákon az összes program rendelkezik három alapvető I/O csatornával:
 - *Standard input*: ezen keresztül adatokat tudunk a programba bevinni. A „külső” vége általában a felhasználói terminál billentyűzetére van rákötve, de ez az operációs rendszerből módosítható.
 - *Standard output*: ezen keresztül adatokat tudunk kivenni a programból. A „külső” vége általában a felhasználói terminál képernyőjére van rákötve, de ez az operációs rendszerből módosítható.
 - *Standard error*: ezen keresztül a program működésével kapcsolatos információt szokás kijuttatni a programból. A „külső” vége általában a felhasználói terminál képernyőjére van rákötve, de ez az operációs rendszerből módosítható. A különbség a *standard output*-hoz képest, hogy míg az előbbin keresztül a program által kiszámolt adatokat mozgatjuk, addig utóbbin kifejezetten a működéssel kapcsolatos információk, hibák jelennek meg. (Elméletben.)

- A standard csatornákat a Python szöveges fájlként kezeli, ezek a `sys` modulban érhetőek el (a modul importja után).
 - *Standard input:* `sys.stdin`
 - *Standard output:* `sys.stdout`
 - *Standard error:* `sys.stderr`
- A Python alapvető beviteli és kiviteli utasításai, az `input` és `print` függvények alapvetően a *standard input* és *standard output* csatornákkal dolgoznak, de a `print` hívásánál a *standard error* is beállítható a `file=sys.stderr` argumentummal.

- Az `input(prompt)` függvényhívás először a *standard output*ra kiírja a `prompt` sztringet, majd a *standard input*ról beolvas egy teljes sort (a sorvége jel nélkül), és szöveggént (`str` típusú érték) visszaadja azt.

```
>>> l = input('|-->_')
|--> text
>>> l
'text'
```

- A `print(*objs, sep=' ', end='\n', file=sys.stdout)` függvényhívás
 - sztringgé konvertálja a paramétereit (`*objs`),
 - összefűzi őket a `sep` értékét (default: `' '`) használva elválasztó karakterként,
 - a végére fűzi az `end` értékét (default: `'\n'`),
 - majd a kapott sztringet kiírja a `file`-ba (default: `'sys.stdout'`).

```
>>> a = 6 * 7
>>> print('A', 'válasz', a, sep=': ', end='!\n')
A: válasz: 42!
```

- Az ASCII, és ezzel együtt a Unicode karaktertáblázat első 32 karaktere úgynevezett „vezérlő” karakter. Ezen karakterek terminálra íráskor nem jelennek meg, hanem a terminál viselkedését befolyásolják.
- Az ilyen karaktereket úgynevezett *escape* szekvenciák segítségével tudjuk megadni (de a sztringben ugyanúgy egyetlen karakterként tárolódnak, mint a többi „normális” karakter):

`\a` Hangjelzés (ha a terminálon engedélyezett).

`\n` Új sor kezdése.

`\r` Visszaugrás az adott sor elejére.

`\b` Egy karakterrel visszább ugrás a sorban.

`\t` Ugrás a következő tabulátor pozícióra.

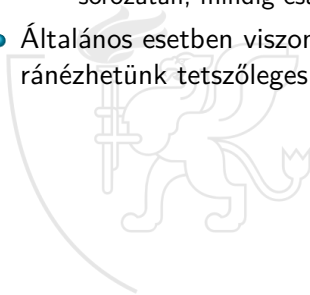
`\v` Soremelés a kurzor vízszintes mozgatása nélkül.

`\f` Lapdobás.

Hozzáférés az adatállományokhoz

Standard csatornák

- A már említett 3 fájl, a `sys.stdin`, `sys.stdout` és `sys.stderr` úgy képzelhető el, mint egy-egy folyam:
 - Mi csak nyitni és zárni tudjuk a „zsilipet”, látjuk mi folyik át, de visszahozni azt már nem tudjuk.
 - Vagy másképpen: mi csak egy irányban (előre) lépegethetünk a fájlban lévő elemek sorozatán, mindig csak a következő elemre.
- Általános esetben viszont egy fájl tartalma megmarad, tehát akárhányszor újra ránézhetünk tetszőleges részére.



Hozzáférés az adatállományokhoz

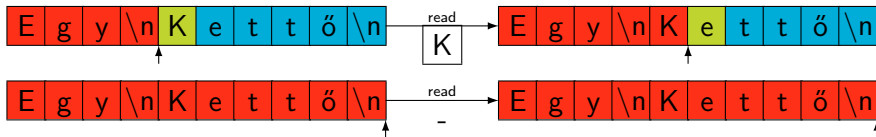
Fájlok

- Minden fájl felfogható elemek (szöveges fájlok esetén például karakterek) sorozataként.
- Ehhez a sorozathoz tartozik egy író-olvasó fej, ami a sorozatot két részsorozatra bontja, tehát egy fájl minden lehetséges értéke megadható $[a_0, \dots, a_{i-1}][a_i, \dots, a_{n-1}]$ alakban.
 - Az író-olvasó fej által kijelölt elem a második sorozat első eleme (vagy a sorozat vége, ha a második sorozat üres sorozat).
- Bármilyen művelet a fájlban csak a fejen keresztül történhet.
- Lehetséges *elemi* műveletek:
 - Olvasás** Kiolvassuk a fej által kijelölt elem értékét és a fej egyet továbblép.
 - Írás** Megváltoztatjuk a fej által kijelölt elem értékét, vagy ha az a fájl vége volt, akkor hozzáfűzzük az új elemet; a fej egyet továbblép.
 - Pozícionálás** A fejet tudjuk mozgatni.

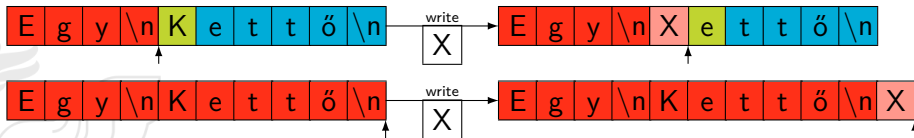
Hozzáférés az adatállományokhoz

Fájlok – elemi műveletek

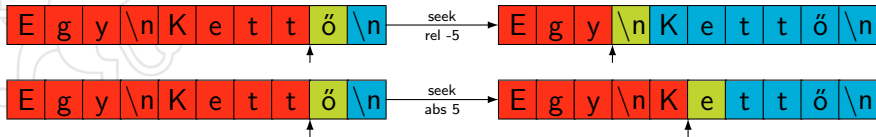
Olvasás



Írás



Pozícionálás



Hozzáférés az adatállományokhoz

Fájlok használata

- A fájl (mint erőforrás, resource) használata következő sémát követi:
 - Először „meg kell nyitni” a fájlt, azaz hozzáférést kell kérnünk az erőforráshoz. Ez python-ban az `open(file, mode="rt", encoding="utf-8")` függvény (és jó gyakorlat szerint a `with` utasítás) segítségével történik.
 - Ezután a visszakapott érték segítségével tudjuk majd használni (írni/olvasni) a fájlt például a `.read()`, `.readline()`, `.readlines()` vagy `.write()` metódusok, vagy akár a `print()` függvény vagy a `for` utasítás segítségével.
 - Végül a `.close()` függvény segítségével „lezárjuk” a fájlt, elengedjük az erőforrást.



- `open(file, mode="rt", encoding="utf-8")`

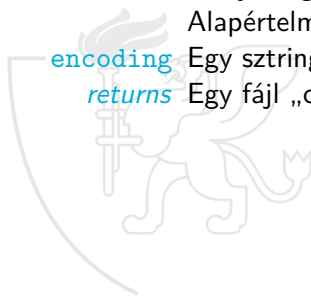
Fájl megnyitása.

file Egy sztring, a fájlrendszeren létező fájlra való (az operációs rendszerben) szabályos hivatkozás.

mode A fájl megnyitási módját leíró sztring. Lehetséges karakterek: `r`, `w`, `a`, `b`, `t`. Alapértelmezés: `'rt'`.

encoding Egy sztring, a fájl karakterkódolását adja meg. Alapértelmezés: `'utf-8'`

returns Egy fájl „objektumot” ad vissza.



- A `mode` sztring karaktereinek értelmezése:
 - 'r' Egy már létező fájl megnyitása olvasásra. Ha nem létezik az hibát okoz.
 - 'w' Egy fájl megnyitása írásra. Ha már létezett, a régi tartalma törlődik, egyébként létre lesz hozva.
 - 'a' Egy fájl megnyitása hozzáfűzésre. Ha már létezett, a régi tartalma megmarad, az új a végéhez fűződik. Ha nem létezett a fájl, akkor egyenértékű a `w`-vel.
 - 't' Fájl szöveges megnyitása. A beolvasás *karakterenként*, a megadott kódolás és az operációs rendszer típusa alapján történik.
 - 'b' Fájl bináris megnyitása. A beolvasás és kiírás *bájtonként* történik, a művelet során nem történik semmilyen konverzió, a fájl és memória tartalma bájról bájtra megegyezik. (Most nem fogunk vele foglalkozni.)

- Egyszerű megnyitás olvasásra:

```
f = open("/etc/passwd", "rt")
```

- Egyszerű megnyitás írásra:

```
f = open("output.txt", "wt")
```

- Ezek után az utasítások után az `f` változón keresztül tudjuk elérni a megadott fájlt. A megoldás hátránya, hogy az esetleges hibákra nem igazán készül fel, így a fájl lezárása, az erőforrás „felszabadítása” nem minden esetben biztosítható.

- Ha azt szeretnénk, hogy a fájl egy esetleges hiba esetén is lezáródjon, akkor használhatjuk a `try` utasítás `finally` blokkját:

```
try:
    f = open("/etc/passwd", "rt")
    ...
finally:
    f.close()
```

- A `try` blokkban az `f` változón keresztül tudjuk elérni a megadott fájlt. Ha menet közben bármilyen futási hiba adódik is, a `finally` blokkban lévő `f.close()` akkor **is** lefut, tehát a fájl lezárása mindenképpen (hibás és hibátlan futás esetén is) megtörténik.

- A Python „jó gyakorlata” fájlok használatára a `with` utasítás/környezet használata:

```
with open("/etc/passwd", "rt") as f:  
    ...
```

- A Python `with` környezete kifejezetten megnyitandó/lezárandó erőforrások egyszerűbb használatára való. Ez automatikusan intézi az erőforrás lezárását/felszabadítását, hiba esetén is. A fájl a blokkon belül az `as` kulcsszó után megadott `f` változón keresztül érhető el.

- A szöveges módban megnyitott fájlokat karakterenként, a bináris módban megnyitott fájlokat bájonként kezeljük. Így az *elemek száma* is ennek megfelelően lesz meghatározva.
- Egy utf-8 kódolású fájlból egy Á karaktert beolvasva például szöveges módban az egyetlen karakteres 'Á' sztringet, míg bináris módban a kettő bájtos b'\xc3\x81' bináris sztringet kapjuk.
- `.read(size=-1)`
Olvasásra megnyitott fájl tartalmának beolvasása.
size A beolvasandó elemek maximális száma. Ha az érték `-1` vagy nincs megadva, akkor a fájl teljes maradék tartalma beolvasásra kerül.
returns A beolvasott elemek sorozata sztringként.

- `.readline(size=-1)`

Olvasásra megnyitott fájl egy sorának beolvasása (a sorvége jellel együtt).

size A beolvasandó elemek maximális száma. Ha az érték `-1` vagy nincs megadva, akkor a teljes következő sor beolvasásra kerül.

returns A beolvasott sor sztringként.

- `.readlines(hint=-1)`

Olvasásra megnyitott fájl összes sorának beolvasása (a sorvége jelekkel együtt).

hint A beolvasandó elemek maximális számára vonatkozó javaslat. Ha a sorok beolvasása során elértük ezt az értéket, akkor több sor nem kerül beolvasásra. Ha az érték `-1` vagy nincs megadva, akkor a fájl teljes maradék tartalma beolvasásra kerül.

returns A beolvasott sorok sztringekként egy listában.

- A megnyitott fájl sorait nem szükséges a `.readline()` segítségével beolvasni, a `for` segítségével végig is lehet iterálni.
 - Soronkénti beolvasás `while` és `readline()` segítségével

```
with open("data.txt", "rt") as file:
    while line := file.readline():
        process(line)
```

- Soronkénti feldolgozás `for` segítségével

```
with open("data.txt", "rt") as file:
    for line in file:
        process(line)
```

- A szöveges módban megnyitott fájlokat karakterenként, a bináris módban megnyitott fájlokat bájtónként kezeljük.

- `.write(s)`

Az `s` sztring kiírása az írásra megnyitott fájlba.

s A kiírandó sztring. Szöveges módban egy normál sztring; a fájlba az adott karakterek kódjai a fájl megnyitáskor meghatározott kódolás (alapértelmezésben *utf-8*) alapján kerülnek kiírásra. Bináris módban egy bináris sztringet vár, a sztring bájtjai változtatás nélkül lesznek kiírva a fájlba.

returns A kiírt elemek (karakterek vagy bájtok) száma.

- A `.write()` csak sztringeket tud kezelni, ami nem gond, de nem mindig kényelmes. Szerencsére a `print()` függvény képes bármilyen (írásra megnyitott) fájlt kezelni.

- `print(*objs, sep=' ', end='\n', file=sys.stdout)`

A megadott argumentumok fájlba írása.

***objs** A kiírandó objektumok. Sztringgé lesznek konvertálva.

sep Az elválasztó szöveg. Az egyes objektumok sztringjeit ezzel a szöveggel választjuk el a kiírásban. Alapértelmezés: `' '`.

end A szöveg lezárása. Ez a szöveg kerül a kiírt sztring végére.
Alapértelmezés: `'\n'`.

file Egy írásra megnyitott fájl. Az összeállított karaktersorozat ebbe a fájlba kerül kiírásra. Alapértelmezés: `sys.stdout`.

returns `None`

- `.seek(offset, whence=SEEK_SET)`

Az író-olvasó fej mozgatása.

offset Egy egész szám, a mozgatás mértéke.

whence Egy egész szám, az `offset` értelmezése.

`SEEK_SET` vagy 0 Abszolút pozíció, a fájl elejétől számolva. Ez az alapértelmezés.

`SEEK_CUR` vagy 1 Relatív pozíció a fej aktuális pozíciójától számolva. A negatív érték a fájl eleje felé történő mozgatás.

`SEEK_END` vagy 2 Abszolút pozíció, a fájl végétől visszafelé számolva.

returns Az író-olvasó fej új abszolút pozíciója a fájlban.

- `.tell()`

Visszaadja az író-olvasó fej pozícióját.

returns Az író-olvasó fej abszolút pozíciója a fájlban.

- Amikor már nem használjuk tovább a fájlt, azt le kell zárni, jelezve az operációs rendszer felé (is), hogy a továbbiakban (egyelőre) nincs szükségünk az erőforrásra. Olvasásra megnyitott fájlok esetén ez inkább formalitás (de szükséges formalitás). Írásra megnyitott fájlok esetén a fájl le nem zárása adatvesztést eredményezhet, és egyes környezetekben (ahol esetleg egy írásra megnyitott fájlhoz senki más nem férhet hozzá, még olvasásra sem) konkrétan akadályozhatja mások munkáját.
- `.close()`
Fájl lezárása.



Fájlműveletek

Példák

```
#!/usr/bin/env python3

from sys import argv

def main():
    try:
        file = open(argv[2], 'rt')
        while line := file.readline():
            if argv[1] in line:
                print(line, end='')
    finally:
        file.close()

if __name__ == "__main__":
    main()
```

```
#!/usr/bin/env python3

from sys import argv

def main():
    with open(argv[1], 'rt') as fin,\
         open(argv[2], 'wt') as fout:
        for line in fin:
            fout.write(line[-2::-1])
            fout.write('\n')

if __name__ == "__main__":
    main()
```

Számsorozat jellemzői, fájlból

Problémafelvetés és specifikáció

- Problémafelvetés

- Korábban készítettünk egy programot, ami meghatározza egy valós számsorozat legkisebb és legnagyobb elemét, valamint a sorozat átlagát.
- Sok tízezer, esetleg több millió adatot kézzel bevinni viszont körülményes.
- Át kellene alakítani a programot úgy, hogy az adatokat egy fájlból vegye!

- Specifikáció

- A probléma inputja egy valós számokból álló sorozat.
- Az adatok egy fájlban vannak, soronként egy szám. A fájl a számokon kívül csak whitespace karaktereket tartalmaz.
- A program argumentuma a fájl neve lesz.
- Az output a sorozat legkisebb és legnagyobb eleme, valamint az átlaga.

Számsorozat jellemzői, fájlból olvasás

minimax_file.py [10–36]

```
10 from sys import argv
11
12 def main():
13     db = 0
14     with open(argv[1], 'rt') as file:
15         szam = file.readline().strip()
16         if szam:
17             s_sum = s_min = s_max = float(szam)
18             db = 1
19         while szam := file.readline().strip():
20             szam = float(szam)
21             s_sum += szam
22             db += 1
23
24             if szam < s_min:
25                 s_min = szam
26             elif szam > s_max:
27                 s_max = szam
28
29     if db == 0:
30         print("Üres számsorozat érkezett.")
31     else:
32         print(f"Minimum={s_min:10.3f} Maximum={s_max:10.3f}")
33         print(f"Az átlag={s_sum/db:10.3f}")
34
35 if __name__ == "__main__":
36     main()
```

- Problémafelvetés
 - Adott egy színes kép. Invertáljuk és fordítsuk meg.
- Specifikáció
 - A kép egyszerű PPM formátumú: az első sora a P3 karakterpár majd egy sorvége jel; utána a kép szélessége, magassága, és a maximális képpontintenzitás (M) van decimális számleírással whitespace karakterekkel elválasztva; majd sorban minden pixelhez (a bal felső saroktól sorfolytonosan) három, a $[0, M]$ intervallumba eső egész érték (szintén decimális számleírással), az adott pixel vörös, zöld és kék színkomponensének értéke. A képfájl bárhol tartalmazhat sortörést.
 - A programnak két argumentuma van, a bemeneti és a kimeneti képfájlok nevei.
 - A kimeneti kép inverz (sötétből világos, világosból sötét) és 180 fokban elforgatott.

Képmanipuláció [1/2]

invert.py [1-27]

```
1#!/usr/bin/env python3
2"""
3Programozás Alapjai előadás példaprogram
4SZTE TTIK, Informatikai Intézet, Szoftverfejlesztés Tanszék
5Gergely Tamás, gertom@inf.u-szeged.hu, 2024-11-20
6cc-by-nc-nd
7
8Egy .ppm kép megfordítása és invertálása
9"""
10from sys import argv
11
12def read_ppm(filename):
13    with open(filename, "rt") as file:
14        if file.readline() != 'P3\n':
15            return None
16        content = [int(n) for n in file.read().split()]
17    pic_w = content.pop(0)
18    pic_h = content.pop(0)
19    pic_m = content.pop(0)
20    pixels = []
21    for x in range(pic_w):
22        row = []
23        for y in range(pic_h):
24            red = (x * pic_h + y) * 3
25            row.append(content[red:red+3])
26        pixels.append(row)
27    return { 'width': pic_w, 'height': pic_h, 'maxint': pic_m, 'pixels': pixels }
```

Képmanipuláció [2/2]

invert.py [29–50]

```
29 def write_ppm(filename, picture):
30     with open(filename, "wt") as file:
31         file.write('P3\n{}\n{}\n'.format(picture['width'], picture['height'], picture['maxint']))
32         for row in picture['pixels']:
33             file.write(' '.join(str(spx) for pix in row for spx in pix) + '\n')
34
35 def invert_picture(picture):
36     max_intensity = picture['maxint']
37     for row in picture['pixels']:
38         for pix in row:
39             for spx in range(3):
40                 pix[spx] = max_intensity - pix[spx]
41         row.reverse()
42     picture['pixels'].reverse()
43
44 def main():
45     p = read_ppm(argv[1])
46     invert_picture(p)
47     write_ppm(argv[2], p)
48
49 if __name__ == "__main__":
50     main()
```